- Урок в прозе: https://proproprogs.ru/ml/ml-postanovka-zadachi-mashinnogo-obucheniya (https://proproprogs.ru/ml/ml-postanovka-zadachi-mashinnogo-obucheniya)
- Телеграм-канал: https://t.me/selfedu_sb (https://t.me/selfedu_sb)
- Телеграм-чат: https://t.me/machine_learning_selfedu (https://t.me/machine_learning_selfedu)

- К. В. Воронцов: https://ya-r.ru/2020/05/07/vorontsov-kurs-mashinnoe-obuchenie-2019-shkola-analiza-dannyh/ (https://ya-r.ru/2020/05/07/vorontsov-kurs-mashinnoe-obuchenie-2019-shkola-analiza-dannyh/)
- Лекции К. В. Воронцова: https://www.youtube.com/watch?v=SZkrxWhI5qM&list=PLJOzdkh8T5krxc4HsHbB8g8f0hu7973fK (https://www.youtube.com/watch?v=SZkrxWhI5qM&list=PLJOzdkh8T5krxc4HsHbB8g8f0hu7973fK)
- Лекции С. Николенко: https://www.youtube.com/watch?v=BPcicsBuRdc&list=PL-_cKNuVAYAWXoVzVEDCT-usTEBHUf4AF (https://www.youtube.com/watch?v=BPcicsBuRdc&list=PL-_cKNuVAYAWXoVzVEDCT-usTEBHUf4AF)
- Курс по Python: https://stepik.org/course/100707/ (https://stepik.org/course/100707/)
- Курс по Python ООП: https://stepik.org/course/116336/ (https://stepik.org/course/116336/)
- Курс по основам теории вероятностей: https://stepik.org/course/136024/ (https://stepik.org/course/136024/)
- Курс по NumPy: YouTube (https://www.youtube.com/playlist?list=PLA0M1Bcd0w8zmegfAUfFMiACPKfdW4ifD) | RuTube (https://rutube.ru/plst/535156)

Постановка задачи машинного обучения

Мы продолжаем курс по машинному обучению. На предыдущем занятии мы ввели понятие обучающей выборки $X^l = \{(x_i, y_i)_{i=1}^l\}$ и признакового пространства в виде матрицы $F = \left\| f_j(x_i) \right\|_{l \times n}$. Будем полагать, что набор этих признаков и подается на вход алгоритмов. Причем, частном случае, когда

$$f_j(x_i) = x_{ij},\ j = 1, 2, \ldots$$

имеем неизменные исходные данные измерений $\{x_i\}$.

Получается, что идеальный алгоритм должен уметь отображать входы $\{x_i\}$ в соответствующие выходы $\{y_i\}$. На уровне математики это можно записать через функциональную зависимость между входами и выходами:

$$y_i = y(x_i)$$

Причем, функциональная в широком смысле слова – это может быть любой алгоритм, связывающий вход с выходом. Но мы эту взаимосвязь не знаем. Целью обучения, как раз и является найти такую модель, **решающую функцию** (decision function), которая бы приближала ответы

$$a_i = a(x_i)$$

к требуемым $\{y_i\}$ на всем множестве возможных входных данных X (не только для обучающей выборки, но для всех возможных наблюдений той же природы). Это и есть общая постановка задачи машинного обучения.

Конечно, в таком виде совершенно непонятно, как ее решать, как искать правило преобразования $a(x)$? Необходима конкретизация. По сути, весь курс машинного обучения – это и есть различные вариации решения поставленной задачи.

Как можно ее решить? Наверное, одним из самых простых подходов (и наиболее часто используемых), представить функционал $a(x)$ в виде некоторой выбранной нами параметрической функции:

$$a(x) = g(x, \theta)$$

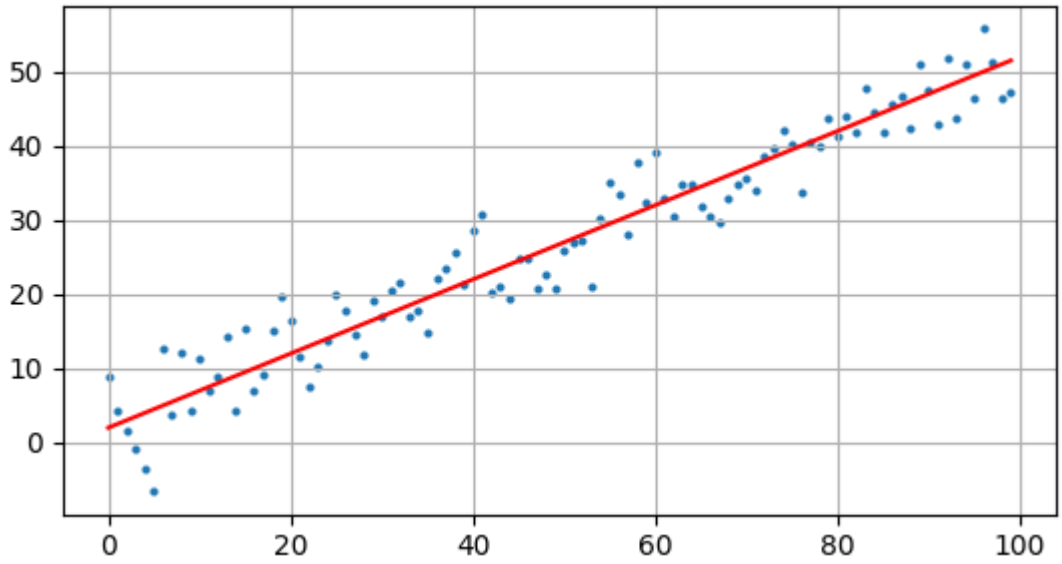
с настраиваемым набором параметров θ. То есть, мы сводим задачу обучения к поиску неизвестных параметров θ и делаем это (в самом простом, но распространенном случае) по обучающей выборке. Причем вид самой функции $g(x, \theta)$ может быть сколь угодно сложным (в математическом смысле) и в общем случае состоять из композиции других, более простых функций. То есть, вид функции $g(x, \theta)$ должен отражать характер (природу, модель) изменения данных между входом и выходом, а параметры θ подгоняют ее под конкретный набор данных.

Чтобы все это было понятнее, давайте рассмотрим классический пример такой задачи – линейную регрессию, когда входы и выходы имеют ярко выраженную функциональную зависимость вида:

$$y_i = kx_i + b + \varepsilon_i,\ i = 1, 2, \dots$$

(здесь $\{\varepsilon_i\}$ - гауссовский (нормальный) шум с нулевым средним и некоторой небольшой дисперсией). Так как мы не можем прогнозировать случайные отклонения, то самое разумное описать модель данных в виде линейной функции с двумя неизвестными параметрами k, b:

$$g(x, k, b) = kx + b$$



В результате, мы имеем вектор параметров $\theta = [k, b]^T$, которые определяют конкретный накло и сдвиг линейной функции. То есть, исходная функция $g(x, k, b)$ описывает весь класс прямых, а при конкретных $\theta = [k, b]^T$ получаем определенную прямую для данных обучающей выборки.

Вот принцип параметрической оптимизации, который расширяется на произвольные функциональные зависимости выходов от входов.

Этап обучения

Хорошо, решающая функция $a(x) = g(x, \theta)$ в виде параметрической функции, определена. Как теперь нам найти значения параметров θ на множестве входов и выходов $X \times Y$ обучающей выборки? Очевидно, они должны быть подобраны так, чтобы уменьшить ошибки между заданными выходами $y(x)$ и теми, которые получаются в нашей модели $a(x)$:

$$a(x) - y(x), \ \forall x$$

Но сама по себе ошибка в качестве оптимизируемой величины не очень удобна, т.к. в точке минимума (нуля) она не образует точки экстремума. Математически было бы лучше использовать функцию, которая бы возрастала с увеличением ошибки и убывала бы с ее уменьшением. Например, можно выбрать, следующие:

- $L(a, x) = |a(x) - y(x)|$ - абсолютная ошибка;
- $L(a, x) = (a(x) - y(x))^2$ - квадратичная ошибка.

Подобные функции получили название **функций потерь** $L(a, x)$ (loss function), которые, фактически, вычисляет меру потерь (несоответствия) между нашей моделью и обучающей выборкой. Конечно, таких функций огромное количество и с некоторыми из них мы будем знакомиться по мере прохождения этого курса.

Однако, сама по себе функция потерь – это случайная величина, которая зависит от алгоритма $a(x)$ и текущего входного вектора x. Поэтому оптимизировать одно какое-то конкретное значение функции потерь – неправильно. Нужно сделать так, чтобы на всем обучающем

множестве, в среднем, ошибка была бы минимальна. В результате мы приходим к понятию **среднего эмпирического риска**:

$$Q(a, X^l) = \frac{1}{l}\sum_{i=1}^{l} L(a, x_i)$$

(Здесь $X^l = \{(x_i, y_i)_{i=1}^l\}$ - обучающее множество). Фактически, это и есть показатель качества, который нужно минимизировать, подбирая значения вектора параметров θ.

Например, если вернуться к нашей задаче линейной регрессии и в качестве функции потерь выбрать квадрат ошибки, то получим функционал качества в виде:

$$Q(a, X^l) = \frac{1}{l}\sum_{i=1}^{l}(a(x_i) - y(x_i))^2 = \frac{1}{l}\sum_{i=1}^{l}(kx_i + b - y(x_i))^2$$

В данном случае параметры k, b легко вычисляются из решения следующей системы линейных уравнений:

$$\begin{cases} \dfrac{\partial Q(a, X^l)}{\partial k} = \displaystyle\sum_{i=1}^{l}(kx_i + b - y(x_i)) \cdot x_i = 0 \\ \dfrac{\partial Q(a, X^l)}{\partial b} = \displaystyle\sum_{i=1}^{l}(kx_i + b - y(x_i)) = 0 \end{cases}$$

Это известная задача под названием **метод наименьших квадратов** (МНК) и я ее подробно рассматривал в одном из видео:

https://rutube.ru/video/9c126e219ac86fbd4f6734029f7f5f91/
(https://rutube.ru/video/9c126e219ac86fbd4f6734029f7f5f91/)

Если вы с ним не знакомы, то советую посмотреть этот материал.

Итак, резюмируя материал этого занятия, можно отметить следующие четыре пункта:

1. Общая задача машинного обучения ставится как поиск модели $a(x)$, которая наилучшим образом описывает природу зависимости входных данных $\{x_i\}$ и целевых выходных значений $\{y_i\}$.
2. Задачу поиска наилучшей модели часто сводят к задаче параметрической оптимизации функции вида $a(x) = g(x, \theta)$.
3. Для нахождения подходящих параметров θ вводится функция потерь $L(a, x)$ и определяется средний эмпирический риск $Q(a, X^l)$. Минимизируя этот показатель качества, получаем набор параметров θ по обучающей выборке.
4. На основе найденной зависимости $a(x) = g(x, \theta)$ в дальнейшем вычисляются выходные